开发笔记

Ebaina
易百纳技术社区

海思 QT 移植

www.ebaina.com
让AI触手可及，让连接无处不在

易百纳技术社区

版本历史

版本	版本更新说明	责任人	校审人	发布时间
V1.0	初次	易百纳板卡团队	易百纳板卡团队	2025 年 10 月 27 日

免责声明

本文档提供有关南京启诺信息技术有限公司产品的信息，未以明示或暗示，或以禁止发言或其它方式授予任何知识产权许可。本文档所陈述的产品文本及相关软件版权 均属南京启诺信息技术有限公司所有，其产权受国家法律绝对保护，未经本公司授权，其它公司、单位、代理商及个人不得非法使用和拷贝，否则将受到国家法律的严厉制裁。

为了得到最新版本的产品信息，请用户定时访问易百纳技术社区网站平台下载或者与淘宝易百纳技术社区客服联系索取。感谢您的包容与支持。

目录

1. QT 源码下载 4
2. QT 源码编译 4
2.1. 解压 QT 源码 4
2.2. 修改 qmake 4
2.3. configure 配置 5
2.4. 编译与安装 7
3. HIFB/GFBG 适配 7
3.1. 修改 linnuxfb 插件 8
3.2. 编译 10
4. 板端环境部署 12
4.1. 添加板端 QT 运行环境 12
4.2. linuxfb 插件 12
4.3. QT 依赖库 13
4.4. 字库 13
5. 板端测试 QT 程序 13
6. 远程连接编译部署 Qt 程序 15
6.1. 安装 QT Creator 15
6.2. 配置 ssh 连接(板端需要先安装 SSH 服务) 18
6.3. 海思构建套件配置 22
6.4. 远程板端验证 22
6.5. 远程板端调试 24

海思 QT 移植_以 SS928 为例

开发环境为 Ubuntu 22.04 虚拟机，针对不同板端使用对应的交叉编译器：Linux/Ubuntu 板端使用 aarch64-mix210-linux，OpenEuler 板端使用 aarch64-openeuler-linux-gnu。

1. QT 源码下载

官网下载链接：https://download.qt.io/archive/qt/

2. QT 源码编译

2.1. 解压 QT 源码

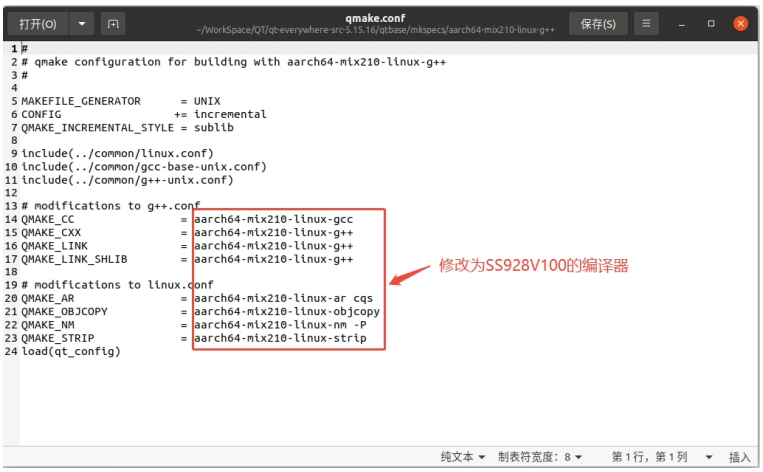
```
tar -xvf qt-everywhere-opensource-src-5.15.16.tar.xz
```

2.2. 修改 qmake

进入 qt-everywhere-src-5.15.16/qtbase/mkspecs/目录，分别为不同交叉编译器创建 mkspec 配置。

①. Linux/Ubuntu 板端使用交叉编译工具 aarch64-mix210-linux

复制一份 linux-aarch64-linux-gnu-g++为 aarch64-mix210-linux-g++，打开文件夹下面文件 qmake.conf，将”aarch64-linux-gnu-”替换为”aarch64-mix210-linux-“，如下图所示：



②. OpenEuler 板端使用交叉编译工具 aarch64-openeuler-linux-gnu

复制一份 linux-aarch64-linux-gnu-g++为 aarch64-openeuler-linux-gnu-g++， 打开文件夹下面文件 qmake.conf，将”aarch64-linux-gnu-” 替换为”aarch64-openeuler-linux-gnu-”， 如下图所示：

2.3. configure 配置

(1) 回到源码根目录， 新建 auto_build.sh 并添加以下配置：auto_build.sh

```
./configure -prefix "$PWD"/SS928V100_QT_INTASLL \
-opensource \
-confirm-license \
-release -strip \
-no-eglfs -linuxfb \
-qt-zlib \
-no-gif \
-qt-libpng \
-qt-libjpeg \
-qt-freetype \
-no-rpath \
-no-pch \
-no-avx \
-no-openssl \
-no-cups \
-no-dbus \
-no-pkg-config \
-no-glib \
-no-iconv \
-xplatform aarch64-mix210-linux-g++ \
-no-opengl \
-nomake examples \
```

```
-nomake tools \
-no-sqlite \
-optimize-size \-skip qtgamepad \
-skip qtandroidextras \
-skip qtmacextras \
-skip qtx11extras \
-skip qtsensors \
-skip qtserialbus \
-skip qtserialport \
-skip qtwebengine \
-skip qtwebchannel \
-skip qtwebsockets \
-skip qtlocation \
-skip qtquickcontrols \
-skip qtpurchasing \
-skip qtconnectivity \
-skip qtscxml \
-skip qtxmlpatterns \
-skip qtnetworkauth \
-skip qtspeech \
-skip qtscript \
-skip qtremoteobjects \
-skip qtcharts \
-skip qtdatavis3d \
-skip qtwebview \
-skip qt3d \
-skip qtquick3d \
-skip qtdoc \
-skip qttranslations \
-skip qtwayland \
-skip qtwebglplugin \
-skip qtwinextras \
-skip qttools
```

注意：以上配置做了一定的裁剪，可按需进行修改。

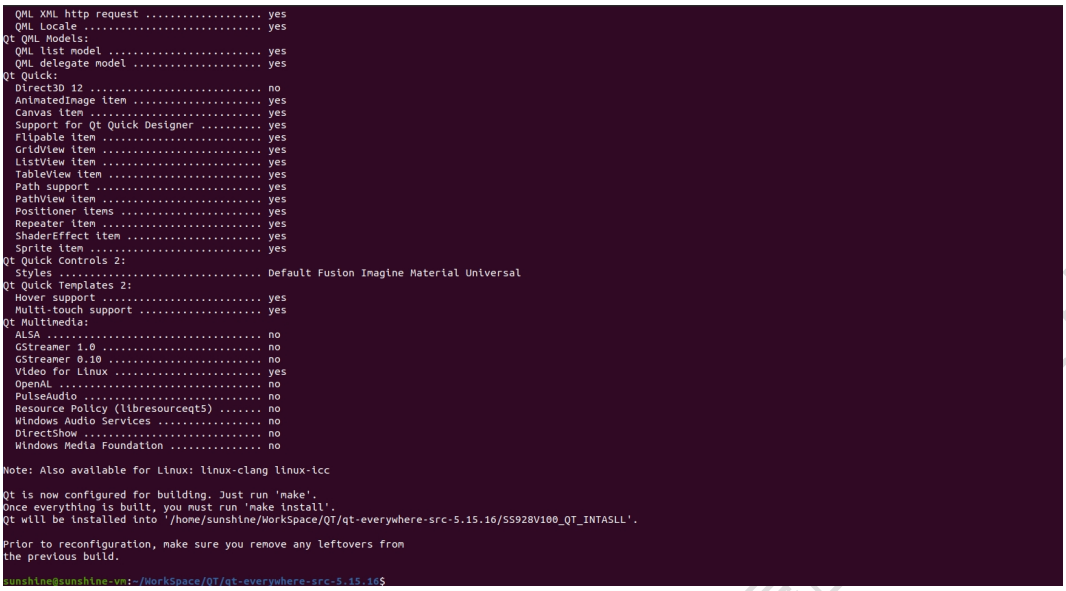
```
-prefix "$PWD"/SS928V100_QT_INTASLL
```

指定安装路径配置为源码目录 SS928V100_QT_INTASLL。

```
-xplatform aarch64-mix210-linux-g++
```

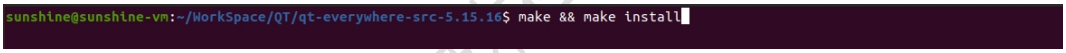
指定编译器为前面配置的 aarch64-mix210-linux-g++,适用于 Linux/Ubunut 板端。如需编译 OpenEuler 版本，将此参数换成 aarch64-openeuler-linux-gnu-g++。

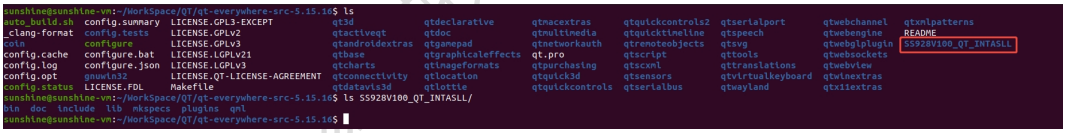
(2) 执行./auto_build.sh，出现以下信息则说明配置没问题。



2.4. 编译与安装

源码目录执行 make && make install





3. HIFB/GFBG 适配

参考文档以及代码：

```
ReleaseDoc\zh\01.software\board\MPP\GFBG 开发指南.pdf
ReleaseDoc\zh\01.software\board\MPP\GFBG API 参考.pdf
SS928V100_SDK_V2.0.2.2/smp/a55_linux/mpp/sample/gfbg/sample_gfbg.c
```

HIFB/GFBG 相关说明可参考以上文档，以下不进行详细描述，只总结以下两个要点：

- HIFB/GFBG 开启前，必须先打开 VO 设备。
- colorkey 功能能让 QT 层设置成配置的颜色时透明，从而达到显示视频层的效果。

3.1. 修改 linnuxfb 插件

(1) 进入源码目录 qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms，复制一份 linuxfb 并改名为 linuxfb_ss928v100

```
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms$ ls
android  cocoa     directfb  haiku      ios      linuxfb_ss928v100  minimal     offscreen  platforms.pro  README  wasm     winrt
bsdfb    direct2d  eglfs     integrity  linuxfb  Makefile           minimalegl  openwfd    qnx            vnc     windows  xcb
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms$
```

(2) 修改 qlinuxfbscreen.cpp 代码进行以下修改：

a. 设置 FB 输出大小

```
#include <linux/fb.h>

#if 1 //add by sunshine
#include "securec.h"
#include "ot_type.h"
#include "ot_common_sys.h"
#include "ss_mpi_sys.h"
#include "gfbg.h"
#include "ss_mpi_vo.h"
#include "ot_common_vo.h"


static struct fb_bitfield g_r16 = {10, 5, 0};
static struct fb_bitfield g_g16 = {5, 5, 0};
static struct fb_bitfield g_b16 = {0, 5, 0};
static struct fb_bitfield g_a16 = {15, 1, 0};

static struct fb_bitfield g_a32 = {24, 8, 0};
static struct fb_bitfield g_r32 = {16, 8, 0};
static struct fb_bitfield g_g32 = {8, 8, 0};
static struct fb_bitfield g_b32 = {0, 8, 0};

#define FHD_WIDTH  1920
#define FHD_HEIGHT 1080

static ot_fb_buf g_canvas_buf;
static td_phys_addr_t g_canvas_addr;
static td_void* g_buf_addr = NULL;
#endif

QT_BEGIN_NAMESPACE
```

FB输出大小

b. 在 initialize 函数中添加海思相关 HIFB/GFBG 配置，相关代码参考于 SS928V100_SDK_V2.0.2.2/sample_gfbg.c

```
#if 1 //add by sunshine

    td_bool is_show;
    td_bool is_compress;
    ot_fb_point point = {0, 0};
    td_u32 byte_per_pixel = 0;
    ot_size fb_size;
    ot_fb_color_format color_format = OT_FB_FORMAT_ARGB1555;    //OT_FB_FORMAT_ARGB1555 OT_FB_FORMAT_ARGB8888

    /* 1. disable HIFB show */
    is_show = TD_FALSE;
    if (ioctl(mFbFd, FBIOPUT_SHOW_GFBG, &is_show) < 0) {
        qErrnoWarning(errno, "FBIOPUT_SHOW_GFBG failed!");
        return false;
    }

    /* 2. set the screen original position */
    if (ioctl(mFbFd, FBIOPUT_SCREEN_ORIGIN_GFBG, &point) < 0) {
        qErrnoWarning(errno, "set screen original show position failed!");
        return false;
    }
```

减少数据量，采用ARGB1555

```
    /*********************************************************************
    *是否过滤颜色 (0x000000) RGB 黑色
    *如果不过滤颜色，也要设置 FBIOPUT_ALPHA_GFBG， 疑似SS928V100 BUG
    *可以使用 FBIOGET_ALPHA_GFBG 获取结构体alpha，其中alpha0默认值与手册里不一样，修改为0XFF
    *********************************************************************/
#if 1
    ot_fb_alpha alpha;
    ot_fb_colorkey color_key;

    memset(&alpha, 0, sizeof(alpha));
    alpha.alpha_en = TD_TRUE;
    alpha.alpha_chn_en = TD_FALSE;
    alpha.alpha0 = 0xFF;
    alpha.alpha1 = 0;
    alpha.global_alpha = 0xFF;
    if (ioctl(mFbFd, FBIOPUT_ALPHA_GFBG, &alpha) < 0) {
        qErrnoWarning(errno, "put alpha info failed!");
        return false;
    }

    color_key.enable = TD_TRUE;
    color_key.value = 0x000000;
    if (ioctl(mFbFd, FBIOPUT_COLORKEY_GFBG, &color_key) < 0) {
        qErrnoWarning(errno, "FBIOPUT_COLORKEY_GFBG !");
        return false;
    }
#else
    ot_fb_alpha alpha;

    memset(&alpha, 0, sizeof(alpha));
    alpha.alpha_en = TD_TRUE;
    alpha.alpha_chn_en = TD_FALSE;
    alpha.alpha0 = 0xFF;
    alpha.alpha1 = 0xFF;
    alpha.global_alpha = 0xFF;
    if (ioctl(mFbFd, FBIOPUT_ALPHA_GFBG, &alpha) < 0) {
        qErrnoWarning(errno, "put alpha info failed!");
        return false;
    }
#endif
```

过滤RGB颜色，即黑色

```
    // mmap the framebuffer
#if 0  //add by sunshine
    mMmap.size = finfo.smem_len;
    uchar *data = (unsigned char *)mmap(0, mMmap.size, PROT_READ | PROT_WRITE, MAP_SHARED, mFbFd, 0);
    if ((long)data == -1) {
        qErrnoWarning(errno, "Failed to mmap framebuffer");
        return false;
    }

    mMmap.offset = geometry.y() * mBytesPerLine + geometry.x() * mDepth / 8;
    mMmap.data = data + mMmap.offset;
#else
    mMmap.size = finfo.smem_len;

    if (ss_mpi_sys_mmz_alloc(&g_canvas_addr, &g_buf_addr, TD_NULL, TD_NULL, fb_size.width * fb_size.height *
        (byte_per_pixel)) == TD_FAILURE) {
        qErrnoWarning(errno, "allocate memory failed!");
        return false;
    }

    g_canvas_buf.canvas.phys_addr = g_canvas_addr;
    g_canvas_buf.canvas.width = fb_size.width;
    g_canvas_buf.canvas.height = fb_size.height;
    g_canvas_buf.canvas.pitch = fb_size.width * byte_per_pixel;
    g_canvas_buf.canvas.format = color_format;
    if (memset_s(g_buf_addr, fb_size.width * fb_size.height * (byte_per_pixel), 0x00, g_canvas_buf.canvas.pitch * g_canvas_buf.canvas.height) != EOK) {
        qErrnoWarning(errno, "memset_s failed!");
        ss_mpi_sys_mmz_free(g_canvas_addr, g_buf_addr);
        g_canvas_addr = 0;
        return false;
    }

    mMmap.offset = geometry.y() * mBytesPerLine + geometry.x() * mDepth / 8;
    mMmap.data = (uchar*)g_buf_addr + mMmap.offset;
#endif
```

屏蔽原有的MMAP操作

修改为画布操作

c. 绘制函数 doRedraw 中修改为对 HIFB/GFBG 画布更新

```
595  QRegion QLinuxFbScreen::doRedraw()
596  {
597      QRegion touched = QFbScreen::doRedraw();
598
599      if (touched.isEmpty())
600          return touched;
601
602      if (!mBlitter)
603          mBlitter = new QPainter(&mFbScreenImage);
604
605      mBlitter->setCompositionMode(QPainter::CompositionMode_Source);
606      for (const QRect &rect : touched) {
607          mBlitter->drawImage(rect, mScreenImage, rect);
608      }
609
610  #if 1 //add by sunshine
611      g_canvas_buf.update_rect.x = 0;
612      g_canvas_buf.update_rect.y = 0;
613      g_canvas_buf.update_rect.width = g_canvas_buf.canvas.width;
614      g_canvas_buf.update_rect.height = g_canvas_buf.canvas.height;
615      if (ioctl(mFbFd, FBIO_REFRESH, &g_canvas_buf) < 0) {
616          qErrnoWarning(errno, "REFRESH failed !");
617      }
618  #endif
619      return touched;
620  }
```

d. 析构函数 QLinuxFbScreen 释放相关资源

```
326  QLinuxFbScreen::~QLinuxFbScreen()
327  {
328      if (mFbFd != -1) {
329  #if 0
330          if (mMmap.data) {
331              munmap(mMmap.data - mMmap.offset, mMmap.size);
332          }
333  #else
334          if (g_canvas_addr && g_buf_addr) {
335              ss_mpi_sys_mmz_free(g_canvas_addr, g_buf_addr);
336              g_canvas_addr = 0;
337              g_buf_addr = NULL;
338          }
339  #endif
340          close(mFbFd);
341      }
342
343      if (mTtyFd != -1)
344          resetTty(mTtyFd, mOldTtyMode);
345
346      delete mBlitter;
347  }
348
```

3.2. 编译

(1) linuxfb.pro 适配

a. 拷贝 HIFB/GFBG 相关库文件和头文件到 qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100 目录下，相关库文件和头文件可从 SDK 目录 SS928V100_SDK_V2.0.2.2/smp/a55_linux/mpp/out 中获取，可以先一次性

复制 lib 和 include 目录

b. 修改 linuxfb.pro 文件

```
TARGET = qlinuxfb_ss928v100

DEFINES += QT_NO_FOREACH

QT += \
    core-private gui-private \
    service_support-private eventdispatcher_support-private \
    fontdatabase_support-private fb_support-private

qtHaveModule(input_support-private): \
    QT += input_support-private

SOURCES = main.cpp \
          qlinuxfbintegration.cpp \
          qlinuxfbscreen.cpp

HEADERS = qlinuxfbintegration.h \
          qlinuxfbscreen.h

qtHaveModule(kms_support-private) {
    QT += kms_support-private
    SOURCES += qlinuxfbdrmscreen.cpp
    HEADERS += qlinuxfbdrmscreen.h
}

INCLUDEPATH += include
unix:LIBS += -lpthread -lm -ldl -Wl,--start-group \
        lib/libss_mpi.a \
        lib/libss_ive.a \
        lib/libss_tde.a \
        lib/libss_hdmi.a \
        lib/libss_voice_engine.a \
        lib/libss_upvqe.a \
        lib/libss_dnvqe.a \
        lib/libsecurec.a \
        -Wl,--end-group

OTHER_FILES += linuxfb.json

PLUGIN_TYPE = platforms
PLUGIN_CLASS_NAME = QLinuxFbIntegrationPlugin
!equals(TARGET, $$QT_DEFAULT_QPA_PLUGIN): PLUGIN_EXTENDS = -
load(qt_plugin)
```

文本编辑器

已测试只需要这些库

c. 需要依赖的相关库

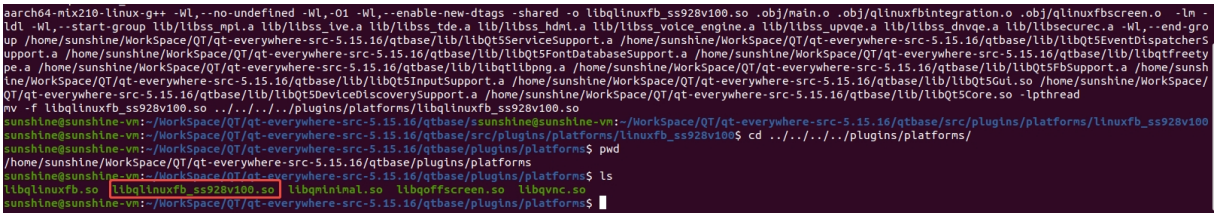
```
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$ ls include/
autoconf.h                  ot_common_cipher.h              ot_common_rc.h          ot_isp_define.h             ss_ivs_md.h                 ss_mpi_motionfusion.h
gfbg.h                      ot_common_dis.h                 ot_common_region.h      ot_ivs_md.h                 ss_mpi_ae.h                 ss_mpi_otp.h
heif_format.h               ot_common_dpu_match.h           ot_common_snap.h        ot_math.h                   ss_mpi_audio.h              ss_mpi_pciv.h
list.h                      ot_common_dpu_rect.h            ot_common_sns.h         ot_mcc_usrdev.h             ss_mpi_avs_conversion.h     ss_mpi_photo.h
npu                         ot_common_dsp.h                 ot_common_svp.h         ot_mipi_rx.h                ss_mpi_avs.h                ss_mpi_region.h
osal_ioctl.h                ot_common_fisheye_calibration.h ot_common_sys.h         ot_mipi_tx.h                ss_mpi_avs_lut_generate.h   ss_mpi_snap.h
osal_list.h                 ot_common_gdc.h                 ot_common_tde.h         ot_motionsensor_chip_cmd.h  ss_mpi_avs_pos_query.h      ss_mpi_sys.h
osal_mmz.h                  ot_common.h                     ot_common_uvc.h         ot_osal.h                   ss_mpi_awb.h                ss_mpi_tde.h
ot_aacdec.h                 ot_common_hdmi.h                ot_common_vb.h          ot_osal_user.h              ss_mpi_cipher.h             ss_mpi_uvc.h
ot_aacenc.h                 ot_common_hnr.h                 ot_common_vdec.h        ot_resample.h               ss_mpi_dpu_match.h          ss_mpi_vb.h
ot_acodec.h                 ot_common_isp.h                 ot_common_venc.h        ot_sns_ctrl.h               ss_mpi_dpu_rect.h           ss_mpi_vdec.h
ot_ao_export.h              ot_common_ive.h                 ot_common_vgs.h         ot_spi.h                    ss_mpi_dsp.h                ss_mpi_venc.h
ot_buffer.h                 ot_common_klad.h                ot_common_video.h       ot_ssp.h                    ss_mpi_fisheye_calibration.h ss_mpi_vgs.h
ot_common_3a.h              ot_common_mau.h                 ot_common_vi.h          ot_type.h                   ss_mpi_gdc.h                ss_mpi_vi.h
ot_common_adec.h            ot_common_mcf_calibration.h     ot_common_vo.h          ot_vdec_export.h            ss_mpi_hdmi.h               ss_mpi_vo.h
ot_common_ae.h              ot_common_mcf.h                 ot_common_vpss.h        ot_vo_export.h              ss_mpi_hnr.h                ss_mpi_vpss.h
ot_common_aenc.h            ot_common_mcf_vi.h              ot_debug.h              ot_vqe_register.h           ss_mpi_isp.h                ss_resample.h
ot_common_af.h              ot_common_md.h                  ot_defines.h            ot_wtdg.h                   ss_mpi_ive.h                ss_vqe_register.h
ot_common_aio.h             ot_common_motionfusion.h        ot_errno.h              pwm.h                       ss_mpi_klad.h               svp_npu
ot_common_avs.h             ot_common_motionsensor.h        ot_i2c.h                securec.h                   ss_mpi_mau.h
ot_common_avs_lut_generate.h ot_common_otp.h                ot_ir.h                 securectype.h               ss_mpi_mcf_calibration.h
ot_common_avs_pos_query.h   ot_common_pciv.h                ot_isp_bin.h            ss_aacdec.h                 ss_mpi_mcf.h
ot_common_awb.h             ot_common_photo.h               ot_isp_debug.h          ss_aacenc.h                 ss_mpi_mcf_vi.h
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$ ls lib
libsecurec.a  libss_dnvqe.a  libss_hdmi.a  libss_ive.a  libss_mpi.a  libss_tde.a  libss_upvqe.a  libss_voice_engine.a
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$
```

进入 qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100 目录，打开终端,执行 ../../../../bin/qmake 即可生成 makefile

```
../../../../bin/qmake      #生成 makefile
```

```
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$ ls
include  linuxfb.json  main.cpp                qlinuxfbdrmscreen.h      qlinuxfbintegration.h  qlinuxfbscreen.h
lib      linuxfb.pro   qlinuxfbdrmscreen.cpp  qlinuxfbintegration.cpp  qlinuxfbscreen.cpp
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$ ../../../../bin/qmake
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$ ls
include  linuxfb.json  main.cpp  qlinuxfbdrmscreen.cpp  qlinuxfbintegration.cpp  qlinuxfbscreen.cpp
lib      linuxfb.pro   Makefile  qlinuxfbdrmscreen.h    qlinuxfbintegration.h    qlinuxfbscreen.h
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/src/plugins/platforms/linuxfb_ss928v100$ make
```

(2) 执行 make 操作，即可在目录 qt-everywhere-src-5.15.16/qtbase/plugins/platforms 生成 libqlinuxfb_ss928v100.so 文件。



注意：OpenEuler 的 linuxfb 适配与上面的操作一致，只需替换交叉编译工具。具体修改见网盘资料的压缩包【易百纳】Euler Pi 2.0\09. 进阶功能开发\03.QT 应用开发\SS928V100_QT.5.15.16_openeuler 移植.tar.xz。

4. 板端环境部署

4.1. 添加板端 QT 运行环境

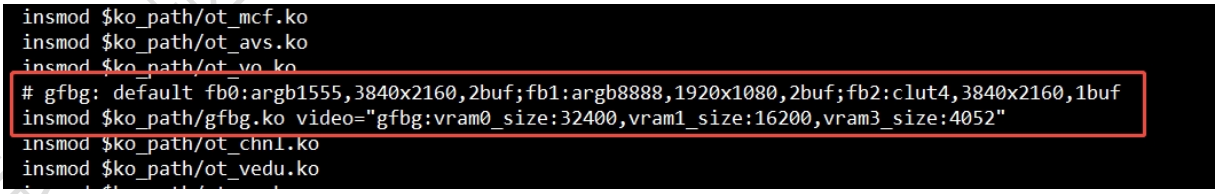
板端/etc/profile 文件中添加 QT 运行环境

```
export QT_QPA_PLATFORM_PLUGIN_PATH=/opt/plugins
export LD_LIBRARY_PATH=/opt/lib/
export QT_QPA_PLATFORM=linuxfb:tty=/dev/fb0
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0:size=1920x1080:offset=0x0:nographicsmodeswitch
export QT_QPA_FONTDIR=/opt/fonts
```

注意：

a). size=1920x1080， 尽量与 libqlinuxfb_ss928v100.so 插件里 FBIOPUT_VSCREENINFO 设置的大小一致；

b). load_ss928v100 中 gfbg.ko 参数配置，fb0 默认按照 3840x2160 2buffer 设置 mmz 内存，若需节省 mmz 内存请自行修改，参考 ReleaseDoc\zh\01.software\board\MPP\GFBG 开发指南.pdf 2.模块加载。

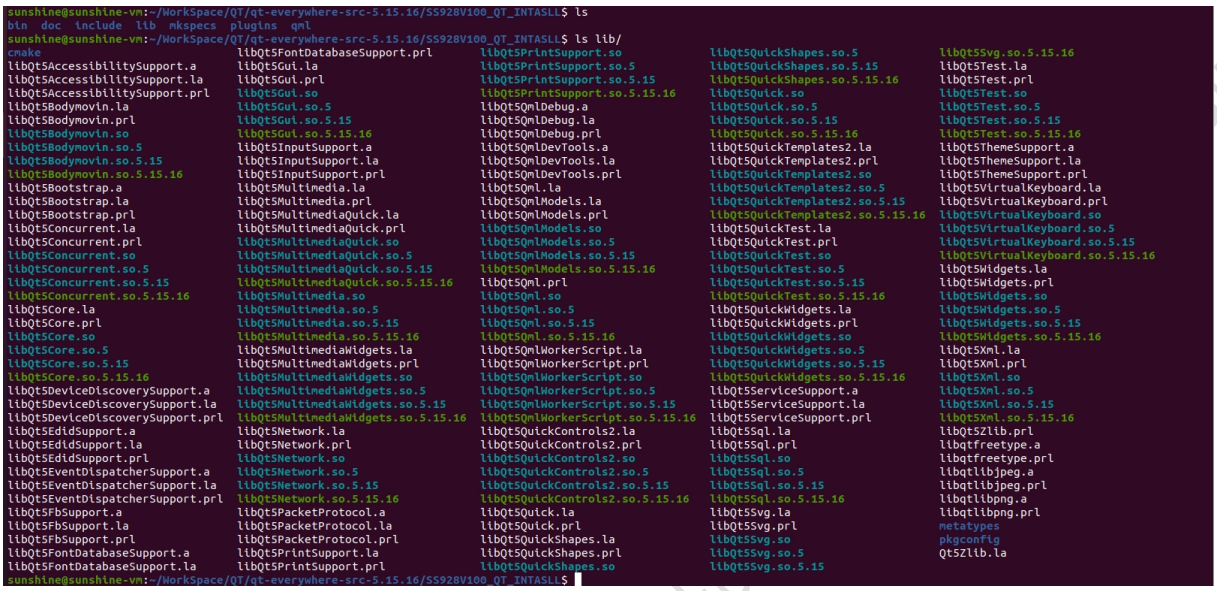


4.2. linuxfb 插件

拷贝 libqlinuxfb_ss928v100.so 插件到板端/opt/plugins 目录下

4.3. QT 依赖库

拷贝 QT 相关库到板端/opt/lib 目录下，仅拷贝测试验证相关库，如果程序中依赖了其他 QT 库，也需要拷贝到该目录下




```
/opt/lib # ls libQt5* -al
-rwxr-x---    1 root     root       5617504 Jul 16 09:31 libQt5Core.so
lrwxrwxrwx    1 root     root            13 Jul 16 09:31 libQt5Core.so.5 -> libQt5Core.so
-rwxr-x---    1 root     root       4832792 Jul 16 09:31 libQt5Gui.so
lrwxrwxrwx    1 root     root            12 Jul 16 09:31 libQt5Gui.so.5 -> libQt5Gui.so
-rwxr-x---    1 root     root       5773864 Jul 16 09:31 libQt5Widgets.so
lrwxrwxrwx    1 root     root            16 Jul 16 09:31 libQt5Widgets.so.5 -> libQt5Widgets.so
/opt/lib #
```

4.4. 字库

拷贝字符文件到板端/opt/fonts 下，OPlusSans3-Medium.ttf 为一加开源免费使用的字库

```
/opt # ls fonts/
OPlusSans3-Medium.ttf
/opt #
```

5. 板端测试 QT 程序

1. 修改并编译 QT 测试程序，以 qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks 为例。

```
77
78  //![16]
79  class QGraphicsRectWidget : public QGraphicsWidget
80  {
81  public:
82      void paint(QPainter *painter, const QStyleOptionGraphicsItem *,
83                 QWidget *) override
84      {
85          // painter->fillRect(rect(), Qt::blue);
86
87          QRectF r = rect();
88          painter->fillRect(r, Qt::blue);
89
90          QRectF innerRect = r.adjusted(5, 5, -5, -5);
91          painter->fillRect(innerRect, Qt::black);
92
93      }
94  };
95  //![16]
96
```

方块颜色为蓝色，在方块里填充黑色方块，即COLORKEY的颜色(过滤该颜色显示透明)

2. 在该目录下打开终端执行../../../../bin/qmake，生成 makefile，执行 make 生成可执行程序 moveblocks

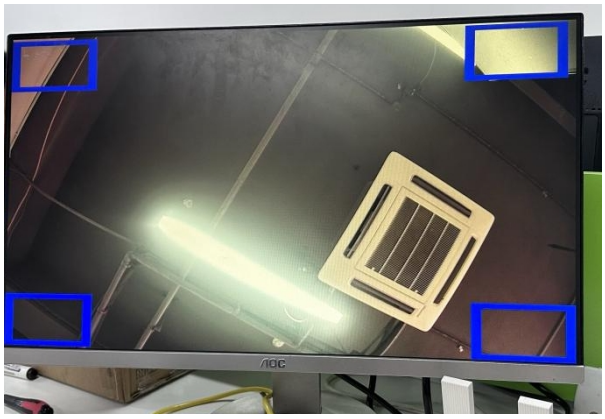
```
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks$ ls
main.cpp  moveblocks.pro
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks$ ../../../../bin/qmake
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks$ ls
main.cpp  Makefile  moveblocks.pro
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks$ make
aarch64-mix210-linux-g++ -c -pipe -O3 -fno-exceptions -Wall -Wextra -D_REENTRANT -fPIC -DQT_NO_LINKED_LIST -DQT_NO_JAVA_STYLE_ITERATORS -DQT_NO_EXCEPTIONS -D_LARGEFILE64_SOURCE -
D_LARGEFILE_SOURCE -DQT_NO_DEBUG -DQT_WIDGETS_LIB -DQT_GUI_LIB -DQT_CORE_LIB -I. -I../../../../include -I../../../../include/QtWidgets -I../../../../include/QtGui -I../../../../i
nclude/QtCore -I.moc -I../../../../mkspecs/aarch64-mix210-linux-g++ -o .obj/main.o main.cpp
aarch64-mix210-linux-g++ -Wl,-O1 -Wl,--enable-new-dtags -o moveblocks .obj/main.o   /home/sunshine/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/lib/libQt5Widgets.so /home/sunshi
ne/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/lib/libQt5Gui.so /home/sunshine/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/lib/libQt5Core.so -lpthread
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks$ ls
main.cpp  Makefile  moveblocks  moveblocks.pro
sunshine@sunshine-vm:~/WorkSpace/QT/qt-everywhere-src-5.15.16/qtbase/examples/widgets/animation/moveblocks$
```

3. 拷贝 moveblocks 到板端， 由于 HIFB/GFBG 的运行依赖 VO 设备的开启，板端需运行相关程序例如 sample_vio 打开 VO 设备。

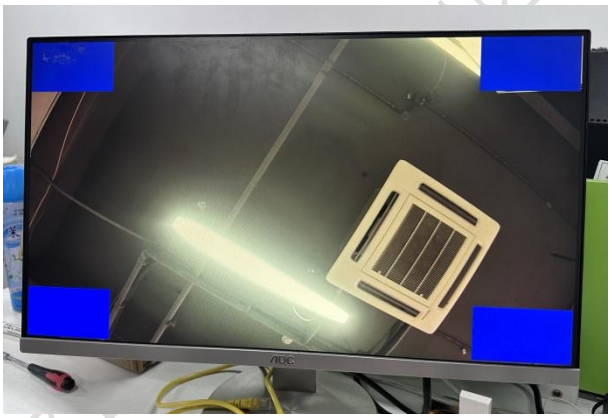
```
/opt/sample/mipi_rx/imx347 # ./sample_vio
usage : ./sample_vio <index> <venc_en>
index:
    (0) one  sensor     4lane sensor0             :vi one  sensor (Online)  -> vpss -> venc && vo.
    (1) one  sensor     4lane sensor1             :vi one  sensor (Online)  -> vpss -> venc && vo.
    (2) two  sensor     4lane sensor(0~1)         :vi two  sensor (offline) -> vpss -> venc && vo.
    (3) four sensor     2lane sensor(0~3)         :vi four sensor (offline) -> vpss -> venc && vo.
venc_en:
    (0) Disable VENC (Video Encoding)
    (1) Enable VENC (Video Encoding)
/opt/sample/mipi_rx/imx347 # ./sample_vio 0 0
linear mode
===IMX347 Slave 4M 30fps 12bit LINE Init OK!===
ISP Dev 0 running !
---------------press enter key to exit!---------------
```

```
/opt/sample/qt # ls
moveblocks
/opt/sample/qt # ./moveblocks
```

fb 插件未过滤颜色



代码修改后



代码修改前

6. 远程连接编译部署 Qt 程序

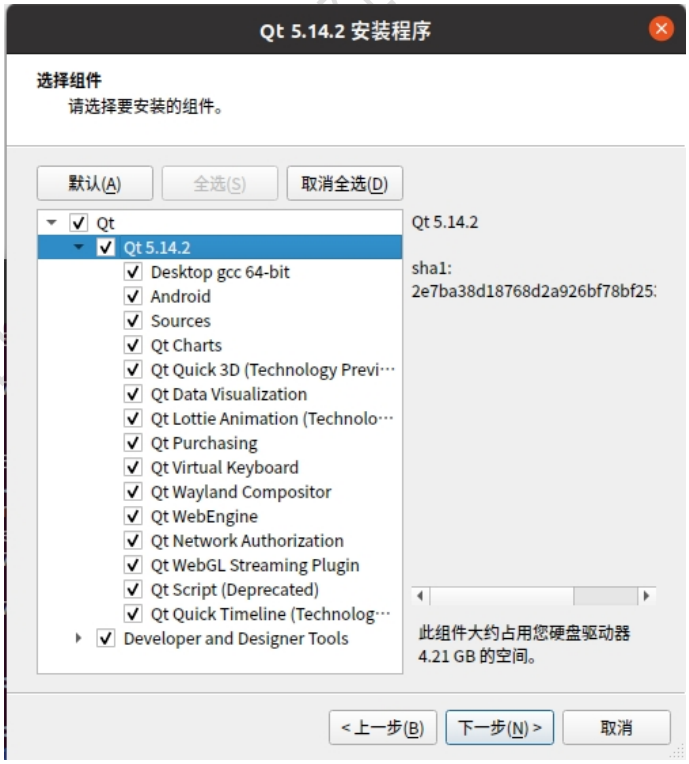
6.1. 安装 QT Creator

建议下载 qt-opensource-linux-x64-5.14.2.run，5.14.2 是最后一个不需要注册账号的版本，需要离线模式下安装。

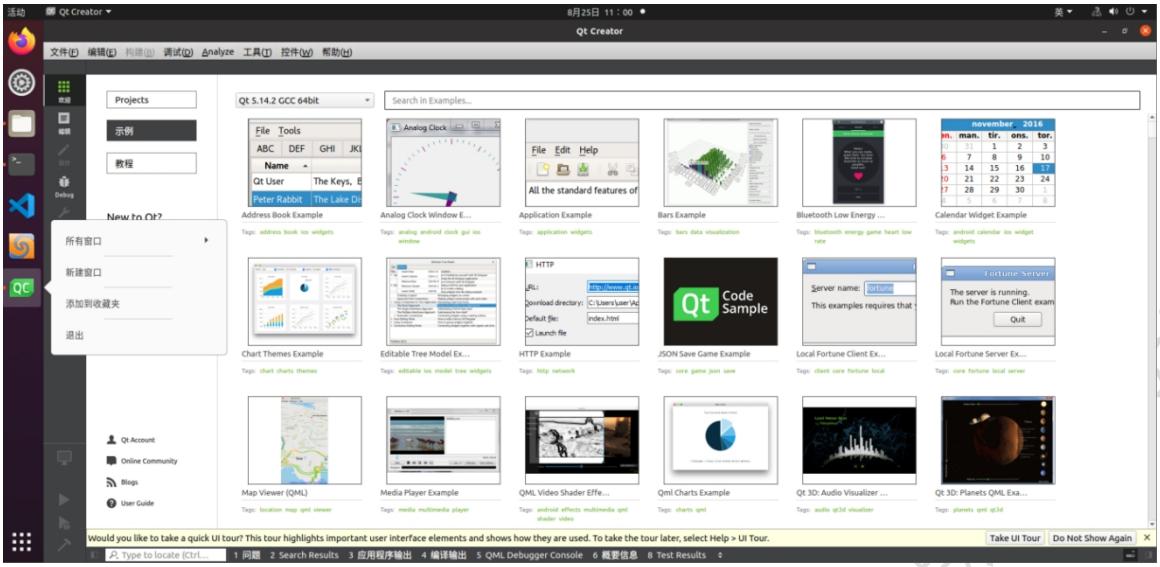
(1) 断网运行，一直下一步。

```
chmod +x ./qt-opensource-linux-x64-5.14.2.run
./qt-opensource-linux-x64-5.14.2.run
```

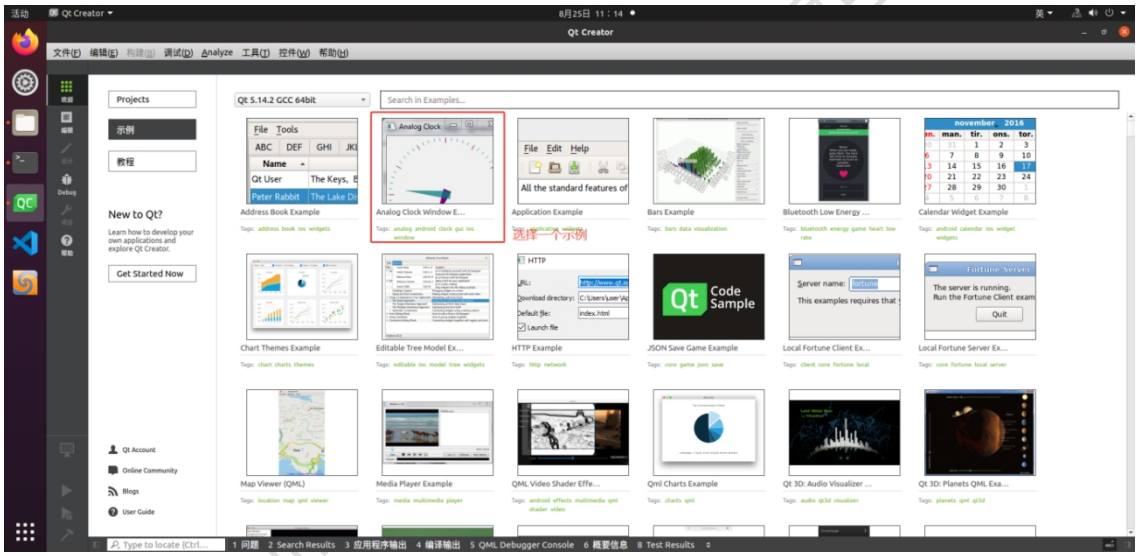
(2) 可以按照自己的需求选择，勾的越多占的硬盘空间越多。安装完成后也可以使用 Maintenance Tool 添加和移除组件。如果不太大，我一般全选，点击下一步。

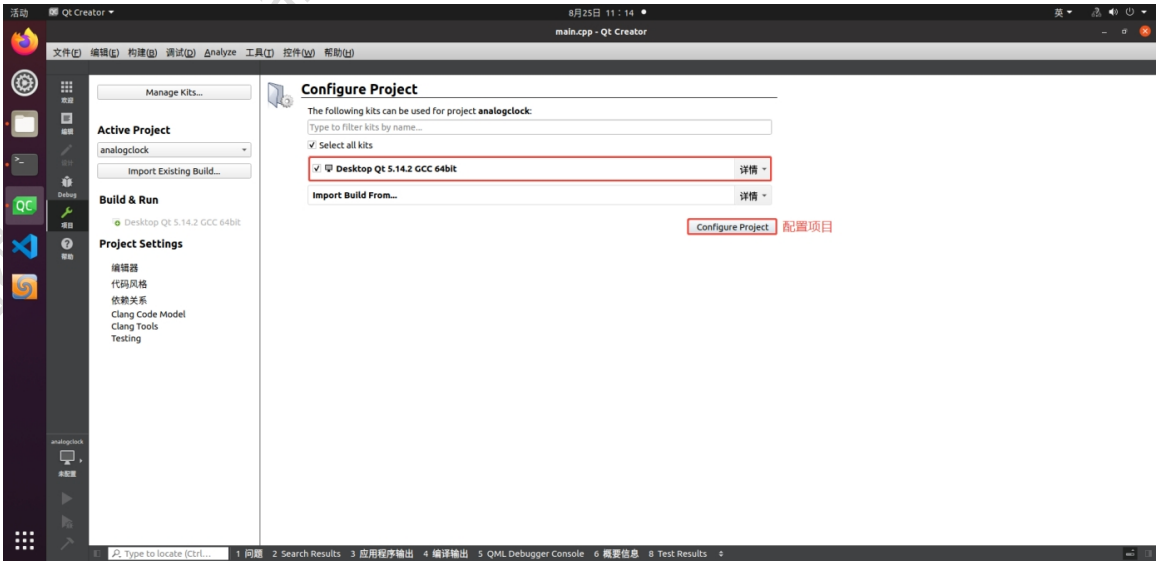


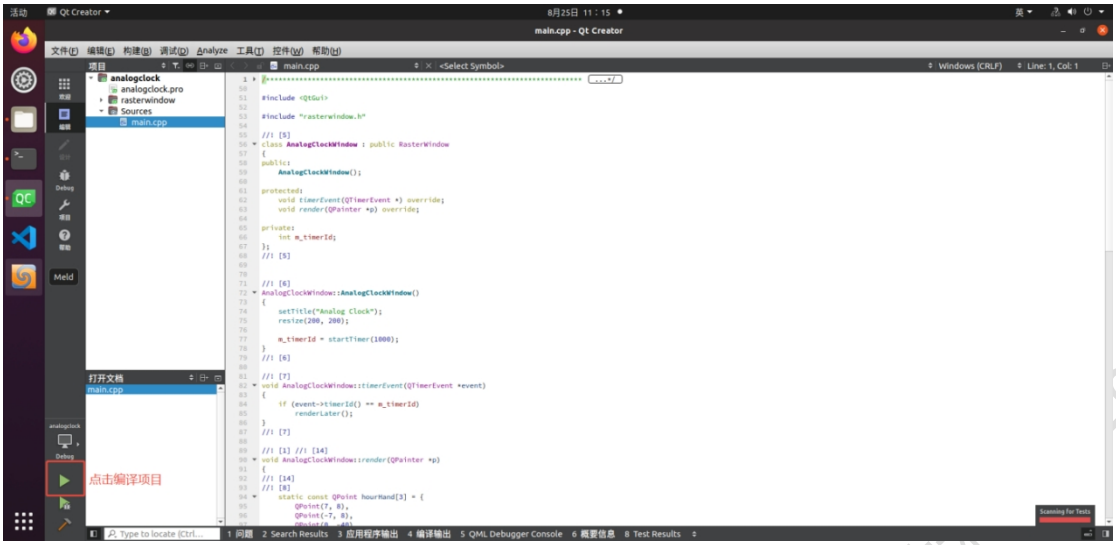
(3) 安装完成会直接弹出界面，点击左边的图标，鼠标右键，点击添加到收藏夹，方便打开。

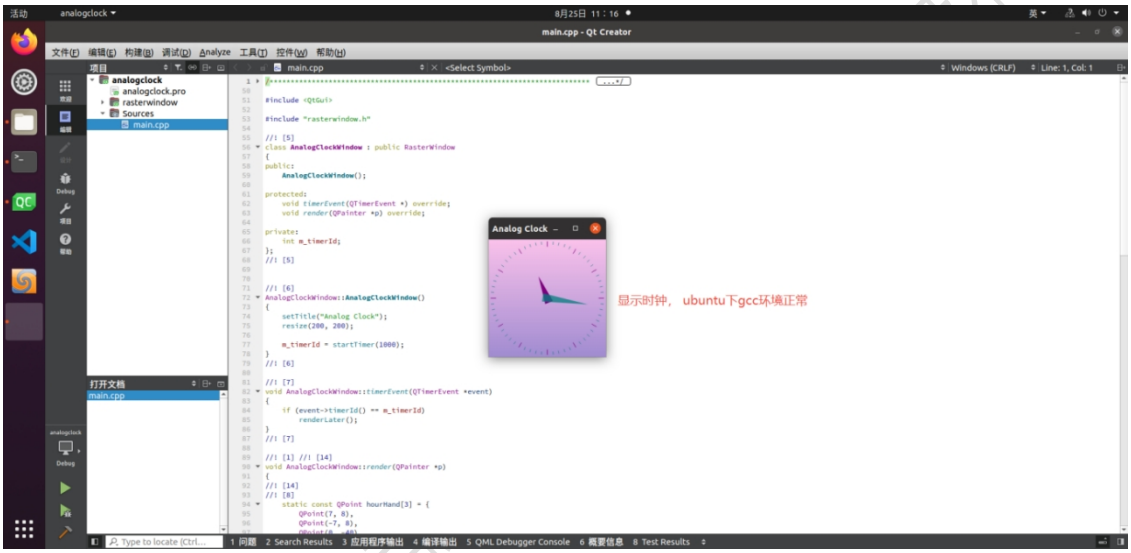


(4) 点击一个示例测试 Ubuntu 下 gcc 构建。



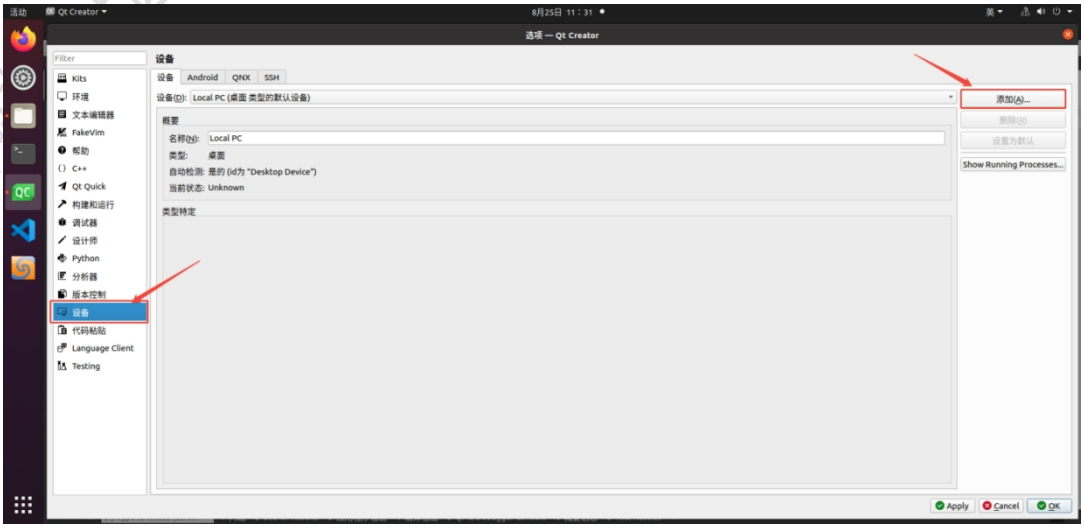






6.2. 配置 ssh 连接(板端需要先安装 SSH 服务)

打开 Qt Creator，点击 工具 > 选项 进入交叉编译的构建套，点击设备，然后右上角点击添加。



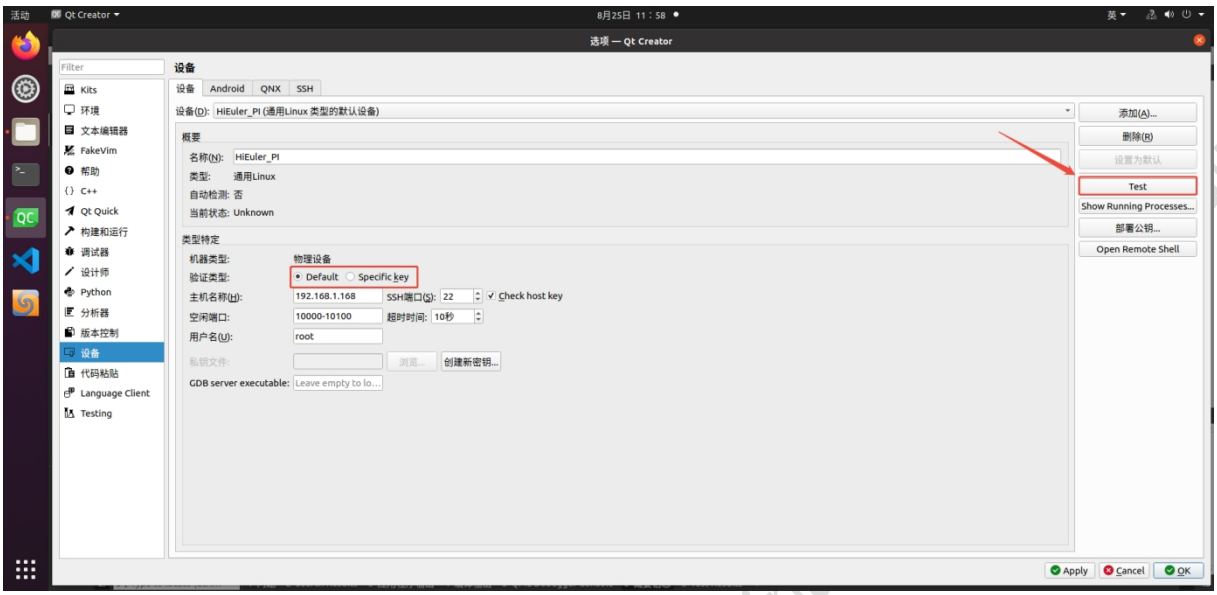
选择种类，点击开始配置向导：

设置名称，输入板子的 IP，和登录的用户名，点击 下一步：

进入密钥部署，一般默认即可，最后点击 完成：

(1) 默认连接

选择验证方式为 Default， 选择 Test 进行测试：

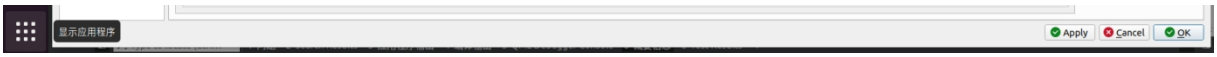


注：如果提示 SFTP 或者 rsync 错误（command not found），可能是板卡没有安装，板卡使用命令：apt install rsync。

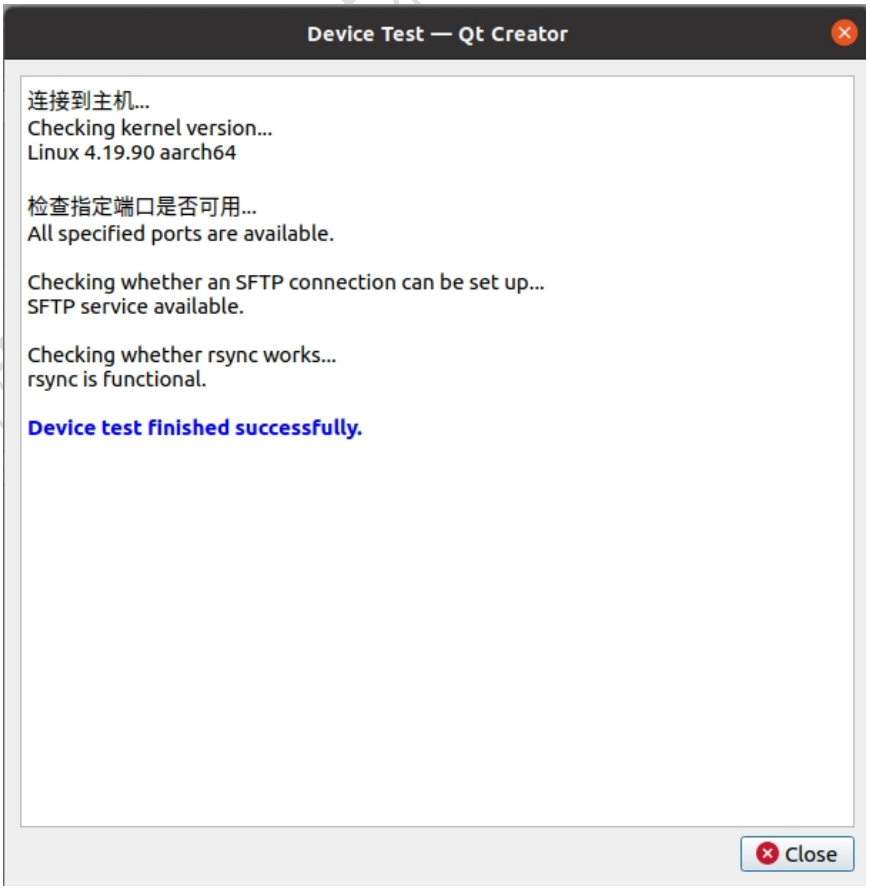
(2) 密钥连接配置

上面配置完成后，就可以正常部署运行，每次 ssh 连接都需要输入密码，接下来讲下使用密钥登

录， 打开 Qt Creator，依次点击 工具 > 选项 > 设备，就可以看到前面配置的 ssh 登录设备：

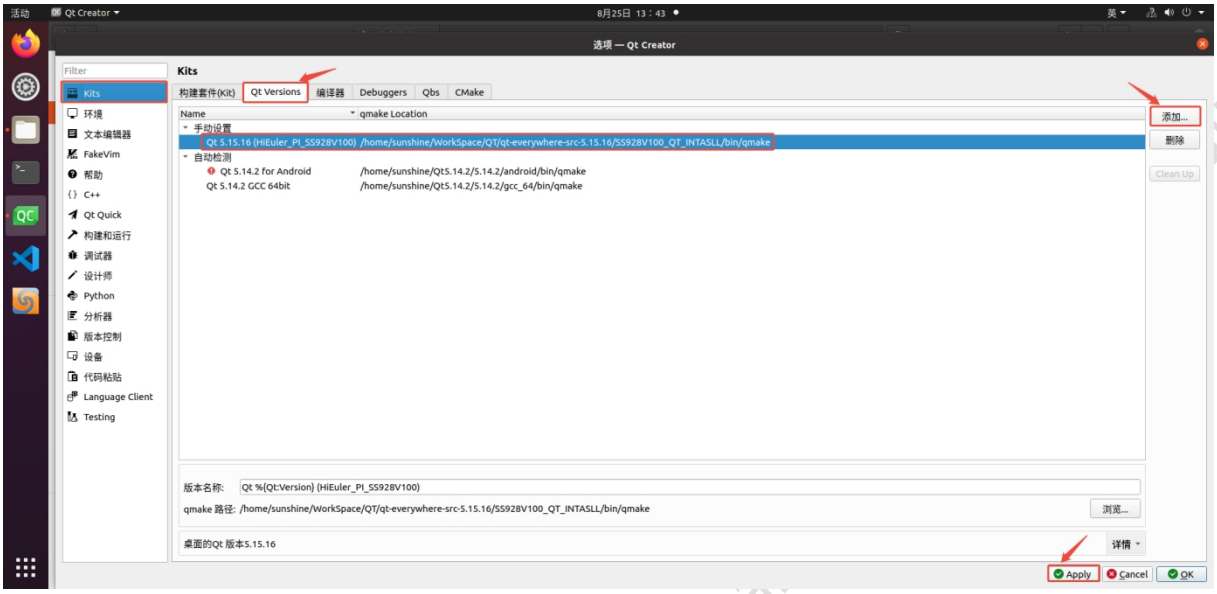


选择验证类型设为 Default，点击右侧部署公钥，输入登录密码，部署成功以后，再将验证类型修改为 Specific_key，再次 TEST 即可成功登录。

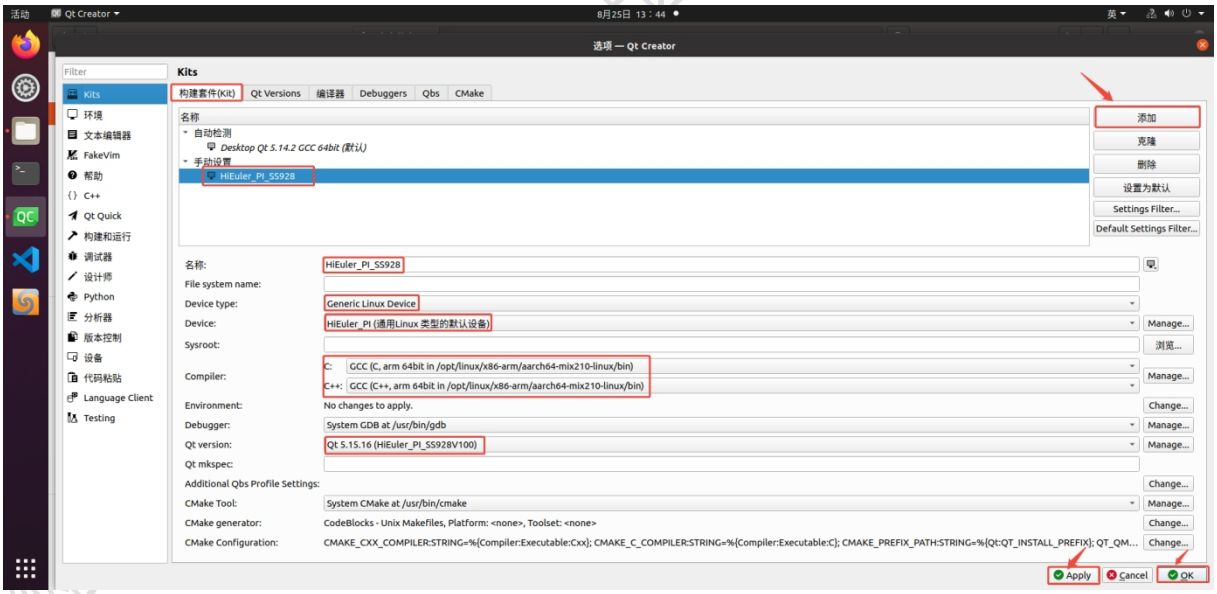


6.3. 海思构建套件配置

选择 Kits，点击 Qt Versions，点击右边添加，选择交叉编译的 SS928V100 的 qmake，点击应用：

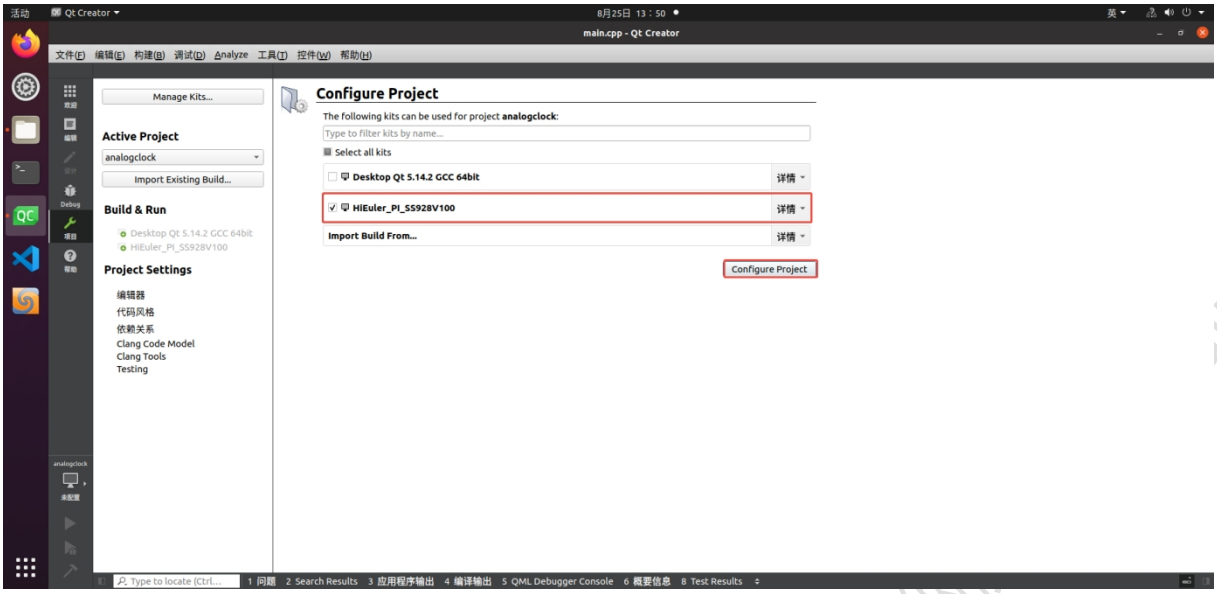


点击构建套件，右边添加构建套件，添加下面内容，当手动设置下的配置不报警告即正常：

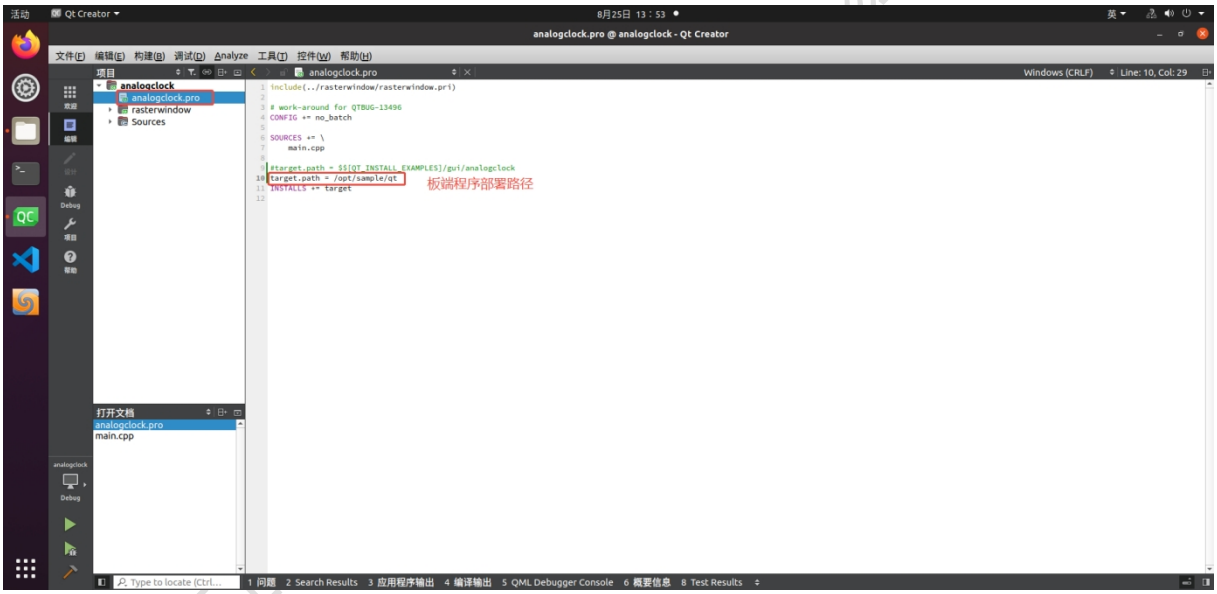


6.4. 远程板端验证

点击一个示例测试选择 HiEuler_PI_SS928V100 构建：

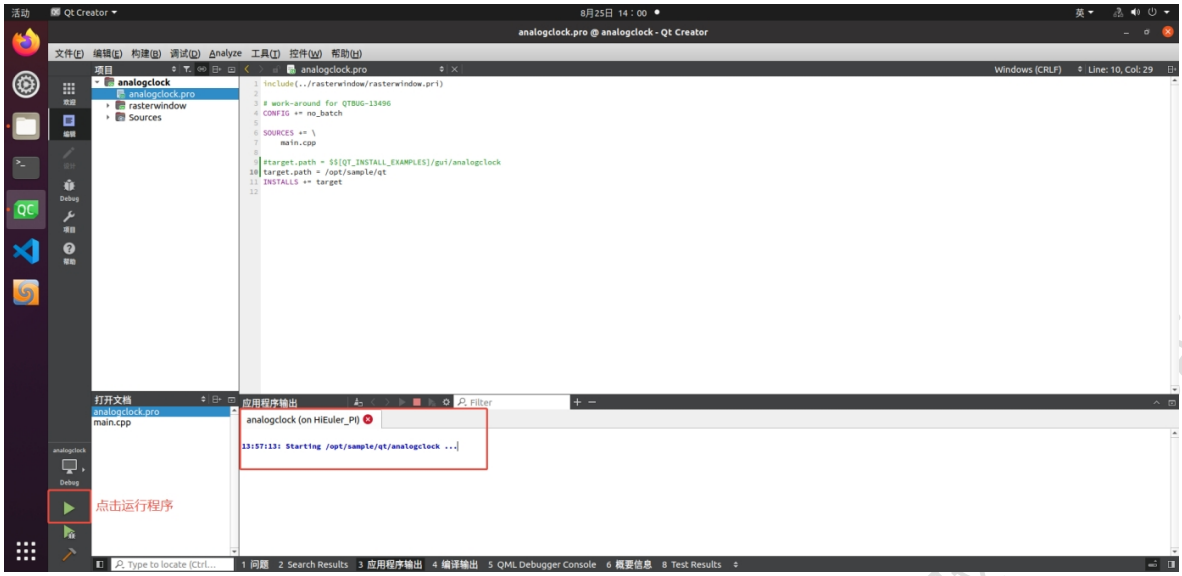


修改 pri 里的板端部署路径：

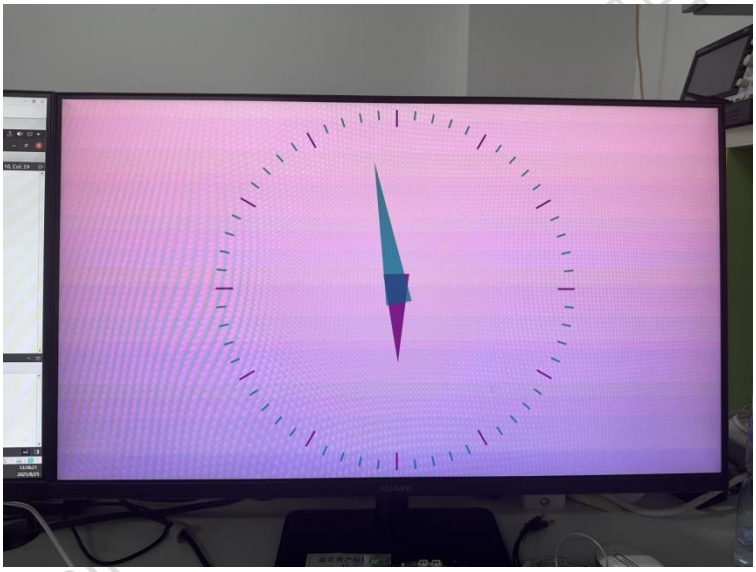


由于 HIFB/GFBG 的运行依赖 VO 设备的开启，板端需运行相关程序例如 sample_vio 打开 VO 设备：

```
root@localhost:/opt/sample/mipi_rx/imx347# ls
sample_vio  sample_vio_4x2lane  sns23_reset_4x2lane.sh
root@localhost:/opt/sample/mipi_rx/imx347# ./sample_vio 0 0
linear mode
===IMX347 Slave 4M 30fps 12bit LINE Init OK!===
ISP Dev 0 running !
--------------press enter key to exit!--------------
```



HDMI 显示：



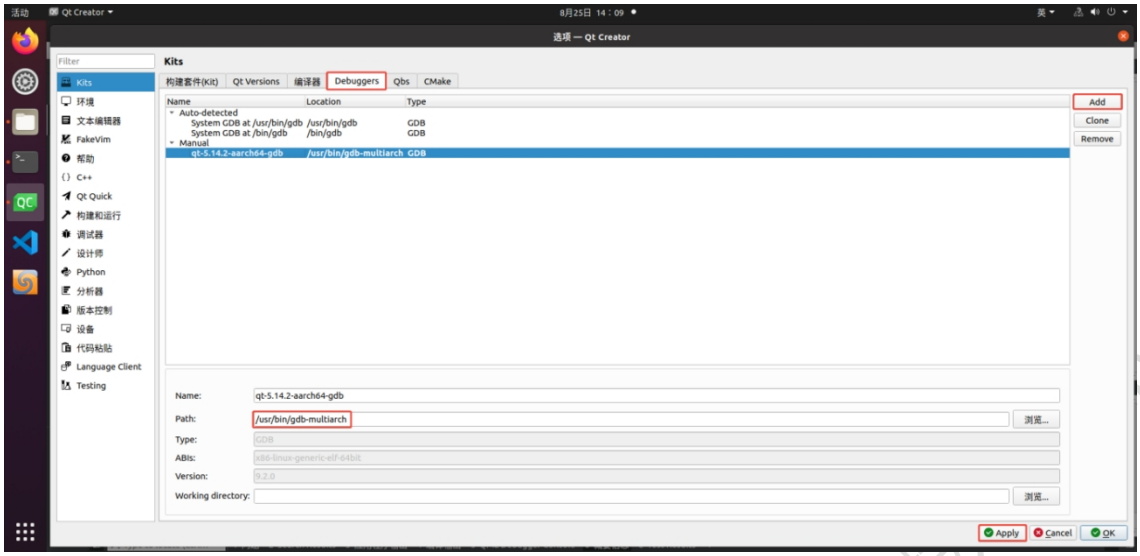
6.5. 远程板端调试

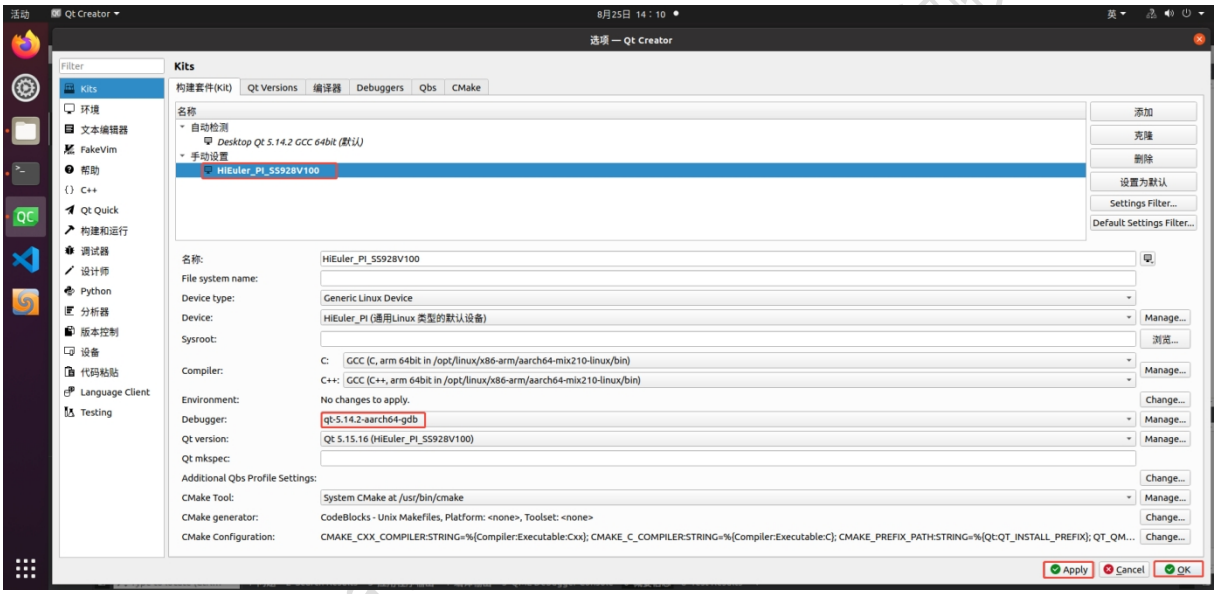
使用上面的例程进行调试，先在 PC 主机端，先使用命令安装 gdb 工具：

```
sudo apt install -y  gdb-multiarch
```

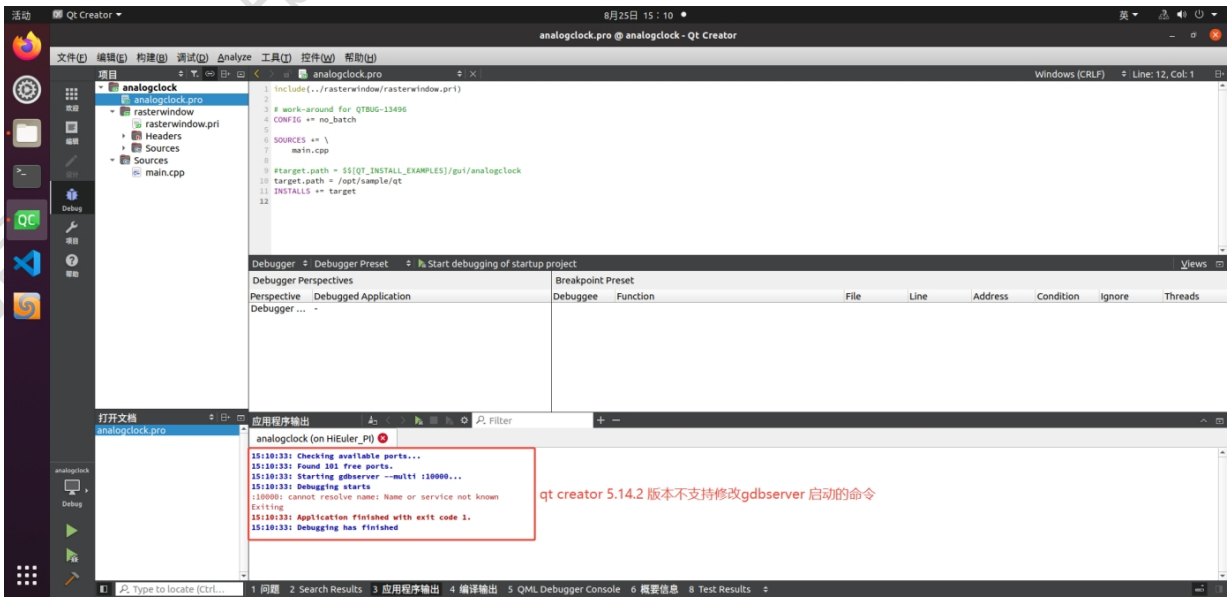
板端(Ubuntu20.04/22.04)安装 gdb 工具， 板端(Linux) 自行移植 gdb 工具：

```
apt install -y gdbserver
```





打断点，直接 F5 调试报错：



(1) 自行尝试最新版 qt creator 测试，菜单栏--> 工具--> 选项--> 设备 ，启动 gdbserver 的命令（Start gdbserver）修改为 gdbserver --multi 0.0.0.0:%port%

(2) 手动启动 gdbserver

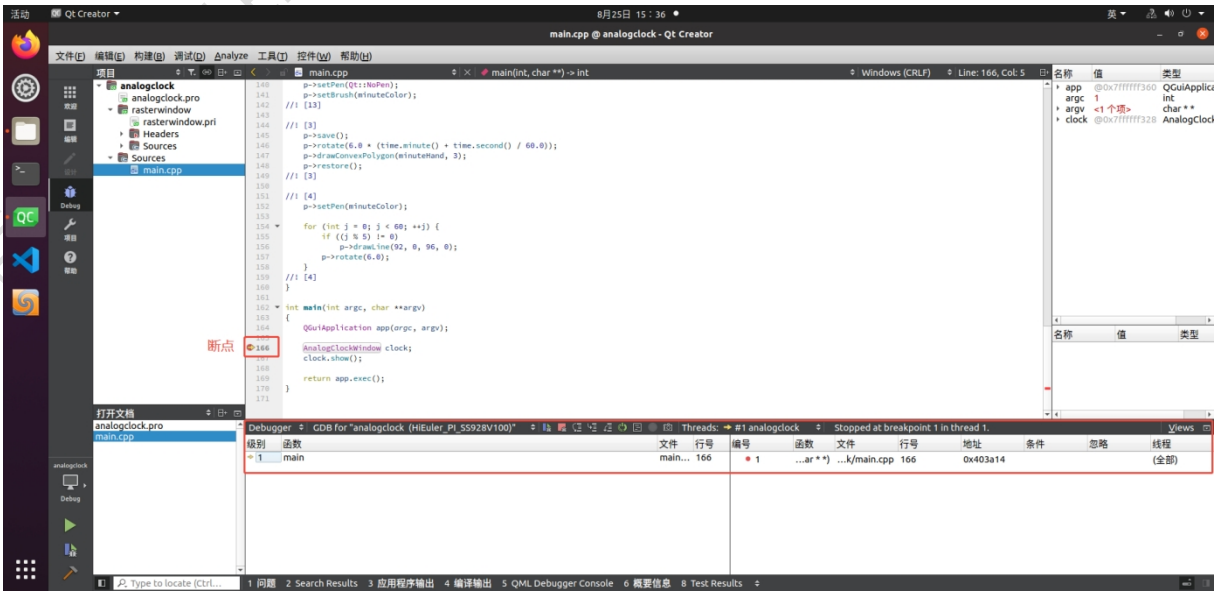
板端运行 gdbserver 命令：

```
root@localhost:/opt/sample/qt# gdbserver 192.168.1.20 10000 ./analogclock
gdbserver: Unable to determine the number of hardware watchpoints available.
gdbserver: Unable to determine the number of hardware breakpoints available.
Process ./analogclock created; pid = 6894
Listening on port 10000
Remote debugging from host 192.168.1.20, port 58702
```

Ubuntu 虚拟机IP

在 Ubuntu 虚拟机里，设置 Qt 单步调试的参数：

Qt Creator 菜单->Debug->Start Debugging->Attach to Running Debug Server



易百纳公司名称：南京启诺信息技术有限公司

易百纳技术社区：www.ebaina.com

技术咨询电话：18013825199

技术服务微信：david089968